

User Guide for CtrlFit

Kui Xie

June 8, 2026

1 Purpose

`ctrlfit` [1] is a `jax-sysid`-based library [2] for training recurrent output-feedback surrogate controllers from trajectories generated by an expert controller-observer pair, such as an NMPC controller coupled with an EKF.

The package provides generic JAX-vectorized expert-data collection, dataset formatting, optional data scaling, `jax-sysid` training setup, optional Lyapunov regularization, parallel-fit model selection, save/load helpers, surrogate simulation, and plotting helpers. The user supplies the surrogate recurrent architecture, plant state and output functions when plant-side operations are needed, expert MPC and EKF behavior, expert trajectories, and any optional custom regularizer.

2 Installation

Install the package from its directory:

```
pip install -e .
```

This also installs the dependencies needed by the CSTR example.

3 Problem Formulation

The core objects are the plant $\mathcal{P}$,

$$\mathcal{P} : \begin{cases} x_{k+1} = f_{\mathcal{P}}(x_k, u_k), \\ y_k = g_{\mathcal{P}}(x_k) + \eta_k, \end{cases} \quad (1)$$

the expert controller-observer pair $\mathcal{C}$,

$$\mathcal{C} : \begin{cases} \chi_k = \mathcal{O}(\chi_{k-1}, y_k, u_{k-1}), \\ \hat{x}_k = h(\chi_k), \\ u_k = \kappa(\hat{x}_k, \bar{y}), \end{cases} \quad (2)$$

and the surrogate controller $\mathcal{S}$

$$\mathcal{S} : \begin{cases} z_{k+1} = f_z(z_k, y_k, \bar{y}; \theta), \\ \hat{u}_k = f_u(z_k, y_k, \bar{y}; \theta). \end{cases} \quad (3)$$

Here, x_k is the plant state, y_k is the measured output, η_k is measurement noise, χ_k is the expert observer state, $\hat{x}_k$ is its state estimate, and $\bar{y}$ is the regulation reference. The surrogate uses its

own recurrent state z_k and produces the fitted control action $\hat{u}_k$. The surrogate is implemented as a user-defined `jax-sysid` recurrent `Model` driven by an output-feedback signal such as

$$w_k = [y_k, \bar{y}].$$

Given expert trajectories

$$\mathcal{D}_t = \left\{ \left\{ (u_k^j, y_k^j, \hat{x}_k^j) \right\}_{k=0}^{N_t^j-1} \right\}_{j=1}^{M_t}, \quad (4)$$

the core supervised objective is

$$\begin{aligned} \min_{\theta} \quad & J(\mathcal{D}_t; \theta) + R_{\text{custom}}(\theta; \mathcal{D}_t), \\ J(\mathcal{D}_t; \theta) = & \frac{1}{M_t} \sum_{j=1}^{M_t} \frac{1}{N_t^j} \sum_{k=0}^{N_t^j-1} \|u_k^j - \hat{u}_k^j\|_2^2. \end{aligned} \quad (5)$$

Only u_k^j and w_k^j are required for plain imitation learning. The observer estimate $\hat{x}_k^j$ is needed by the default Lyapunov regularizer.

3.1 Optional Lyapunov Descent Condition

For optional stability regularization based on [1], let x_s and z_s denote the plant and surrogate equilibrium states. For the shifted closed-loop state

$$\tilde{\xi}_k = \begin{bmatrix} x_k - x_s \\ z_k - z_s \end{bmatrix},$$

define

$$V_k = V(\tilde{\xi}_k) = \|\tilde{\xi}_k\|_P^2 + \eta_L \|\tilde{\xi}_k\|_2^2, \quad (6)$$

$$\hat{S}_k = \hat{S}(\tilde{\xi}_k) = \|\tilde{\xi}_k\|_Q^2 + \eta_Q \|\tilde{\xi}_k\|_2^2, \quad (7)$$

$$S_k = \hat{S}_k + \beta_x \|x_k - x_s\|_2^2 + \beta_z \|z_k - z_s\|_2^2. \quad (8)$$

Here, $\|a\|_M^2 = a^\top M a$, $P, Q \succeq 0$, and $\eta_L, \eta_Q > 0$. For the default $m = 2$ non-monotonic Lyapunov condition, the regularizer penalizes violations of

$$\tau(V_{k+2} - V_k) + (V_{k+1} - V_k) + S_k \leq 0.$$

It also penalizes violations of the hidden-state and control-input equilibrium conditions [1, Sec. III].

4 Main Call

Create the user-defined recurrent model and pass the expert trajectories to `ctrlfit`:

```
from jax_sysid.models import Model
from ctrlfit import ctrlfit

model = Model(
    nx=nz,
    ny=system.nu,
    nu=surrogate_input_dim,
```

```

    state_fcn=surrogate_state_fcn,
    output_fcn=surrogate_output_fcn,
)

model, training_info = ctrlfit(
    model,
    U=U_data,
    Y=Y_data,
    Y_ref=Y_ref_data,
    init_params_fcn=init_fcn,
)

```

This is sufficient for plain supervised fitting. Pass plant-side `state_fcn`, plant-side `output_fcn`, and `X_hat` when using the built-in Lyapunov regularizer:

```

model, training_info = ctrlfit(
    model,
    U=U_data,
    Y=Y_data,
    Y_ref=Y_ref_data,
    init_params_fcn=init_fcn,
    state_fcn=system.state_fcn,
    output_fcn=system.output_fcn,
    X_hat=X_hat_data,
    use_lyap_reg=True,
)

```

5 Data Conventions

<code>model</code>	A user-defined <code>jax_sysid.models.Model</code> . Its inputs are the combined output-feedback signals and its outputs are the surrogate control inputs.
<code>U</code>	Expert controller-observer control trajectories. These are the supervised targets for the surrogate model.
<code>Y</code>	Measured-output trajectories.
<code>Y_ref</code>	Reference trajectories, or a scalar/vector constant. A constant reference is repeated across each measured-output trajectory.
<code>X_hat</code>	Optional observer/state-estimate trajectories. These are required by the default Lyapunov regularizer. A user who intentionally wants to regularize against true plant states may pass those states as <code>X_hat</code> .
<code>state_fcn</code>	Optional plant update function <code>state_fcn(x, u) ↦ x⁺</code> . This is the physical plant-side function, not the surrogate recurrent update. It is required for the built-in Lyapunov regularizer, physical closed-loop validation, and plant rollouts.
<code>output_fcn</code>	Optional plant output function <code>output_fcn(x) ↦ y</code> . This is the physical plant-side function, not the surrogate model output. It is required for the built-in Lyapunov regularizer, physical closed-loop validation, and plant rollouts.

`init_params_fcn` Function `seed` $\mapsto$ `params` for initializing the recurrent-model parameters. It is required for parallel fitting and recommended for reproducible single fits.

Each trajectory signal may be one array of shape (T, dim) or a list of arrays $[\text{traj}_1, \dots, \text{traj}_N]$, where trajectory lengths may differ. The package normalizes the data internally and constructs the surrogate input

$$\mathbf{YY}_{\text{ref}} = \text{concatenate}([\mathbf{Y}, \mathbf{Y}_{\text{ref}}], \text{axis} = -1).$$

6 Training Modes

Plain supervised surrogate training is the default. In this mode, no Lyapunov tail is appended, no Lyapunov regularizer is constructed, no lower bound is added for τ , and `X_hat` is not required:

```
model, training_info = ctrlfit(
    model,
    U=U_data,
    Y=Y_data,
    Y_ref=Y_ref_data,
    init_params_fcn=init_fcn,
    use_lyap_reg=False,
)
```

A user-defined `custom_reg_fcn` may also be passed while `use_lyap_reg=False`. This supports non-Lyapunov regularization.

6.1 Validation and Parallel Fitting

Set `use_parallel_fit=True` to train multiple random initializations and select the best candidate. By default the package uses supervised scoring. Set `closed_loop_validation=True` explicitly to run physical closed-loop validation rollouts. This mode uses plant `state_fcn`, plant `output_fcn`, and `X_hat_val`; it initializes each plant rollout from `X_hat_val[i][0]`, resets the surrogate hidden state to $z_0 = 0$, and scores the resulting plant outputs. If any prerequisite is missing, the package emits a warning and falls back to supervised validation.

```
model, training_info = ctrlfit(
    model,
    U=U_data,
    Y=Y_data,
    Y_ref=Y_ref_data,
    init_params_fcn=init_fcn,
    state_fcn=system.state_fcn,
    output_fcn=system.output_fcn,
    U_val=U_val,
    Y_val=Y_val,
    Y_ref_val=Y_ref_val,
    X_hat_val=X_hat_val,
    use_parallel_fit=True,
    num_parallel_fit=num_parallel_fits,
    select_best_fit="R2",
    closed_loop_validation=True,
)
```

`num_parallel_fit` is the total number of candidate initializations. The optional `n_jobs` argument controls how many worker jobs are allowed to run at the same time during `jax-sysid` parallel fitting and supervised model selection. If `n_jobs=None`, which is the default, `jax-sysid` uses its own default behavior and sets `n_jobs` to `cpu_count()`. To reduce memory pressure while still comparing the same number of candidates, keep `num_parallel_fit` fixed and reduce `n_jobs`:

```
model, training_info = ctrlfit(
    model,
    U=U_data,
    Y=Y_data,
    Y_ref=Y_ref_data,
    init_params_fcn=init_fcn,
    use_parallel_fit=True,
    num_parallel_fit=8,
    n_jobs=2,
)
```

6.2 Optimization Options

The main optimization options passed through to `jax-sysid` are `adam_epochs`, `lbfgs_epochs`, `rho_x0`, `rho_th`, `tau_th`, `train_x0`, and `iprint`. Options may be supplied as keywords or through an options dictionary or `SurrogateTrainingOptions` object.

7 Scaling Convention

Scaling is disabled by default:

```
use_scaling = False
y_mean = 0
y_gain = 1
u_mean = 0
u_gain = 1
```

When `use_scaling=True`, the package scales surrogate inputs and control targets at the package boundary:

$$\text{yyref_mean} = [\text{y_mean}, \text{y_mean}], \quad (9)$$

$$\text{yyref_gain} = [\text{y_gain}, \text{y_gain}], \quad (10)$$

$$\text{YYref_scaled} = (\text{YYref} - \text{yyref_mean})\text{yyref_gain}, \quad (11)$$

$$\text{U_scaled} = (\text{U} - \text{u_mean})\text{u_gain}. \quad (12)$$

The plant-side functions always use physical coordinates. The surrogate functions use training coordinates, and the package boundary converts between the two. During fitting, pass both entries of a mean/gain pair as `None` with `use_scaling=True` to estimate that pair from the training data. During deployment, scaling values must be provided explicitly because online rollout data cannot be used to estimate them.

8 Optional Lyapunov Regularization

Lyapunov regularization is disabled by default. Set `use_lyap_reg=True` to append the default trainable tail

`[z_eq, psi_L, psi_Q, eta_L_raw, eta_Q_raw, tau]`

to the recurrent-model parameters. The package unpacks these final six entries and constructs

$$P = \psi_L^\top \psi_L, \quad Q = \psi_Q^\top \psi_Q, \quad (13)$$

$$\eta_L = \eta_{\min} + \text{softplus}(\eta_{L,\text{raw}}), \quad \eta_Q = \eta_{\min} + \text{softplus}(\eta_{Q,\text{raw}}). \quad (14)$$

The factorization of P and Q , together with the positive η_L and η_Q , makes the functions V and $\hat{S}$ from the problem formulation positive definite. The scalar τ is optimized directly with the hard lower bound $\tau \geq 0$.

The default regularizer is trajectory-based, with descent-penalty weight controlled by `lyap_mu`. It uses `X_hat`, `Y`, and `Y_ref`, re-propagates hidden states from the current model parameters during each loss evaluation, and selects a fixed Lyapunov tube controlled by `lyap_max_trajs`, `lyap_tube_steps`, and `lyap_seed`. When only a subset of training trajectories is used, the package applies farthest-point sampling to the initial `X_hat` states to preserve broad coverage. If `x_eq` or `u_eq` is omitted, the package infers the missing equilibrium value from final expert-trajectory samples and emits a warning.

Advanced users may pass `custom_reg_fcn` to replace the default regularizer or `init_lyap_fcn` to replace the default Lyapunov-tail initializer.

The built-in regularizer currently implements only the two-step condition. The only supported value of `num_lyap_steps` is therefore 2. Other values trigger a warning and are replaced with 2.

9 Training Pipeline

`ctrlfit` performs the following steps:

1. Format the training trajectories and optional validation trajectories.
2. Compute scaling metadata and scale the surrogate-side data when requested.
3. Prepare the optional Lyapunov trajectory tube when `use_lyap_reg=True`.
4. Append the Lyapunov-tail initializer when required.
5. Configure `model.loss(...)` and `model.optimization(...)`.
6. Run either `model.fit(...)` or `model.parallel_fit(...)`.
7. Select the best parallel-fit candidate using validation data when available.
8. Return the fitted model and metadata needed to reuse it.

10 Generic Expert-Data Collection

The package exports a generic fast collector for expert controller-observer rollouts:

```
from ctrlfit import collect_post_initialization_training_data_fast

dataset = collect_post_initialization_training_data_fast(
    state_fcn,
    output_fcn,
```

```

    mpc_fcn,
    ekf_meas_fcn,
    ekf_time_fcn,
    x_ref=x_ref,
    u_ref=u_ref,
    sim_steps=num_sim_steps,
    x0=x0,
    x0_hat=x0_hat,
    P0=P0,
)

```

The collector uses the following callback interfaces:

```

x_next = state_fcn(x, u)
y = output_fcn(x)
u, mpc_state_next = mpc_fcn(x_hat_corr, x_ref, u_ref, mpc_state)
x_hat_corr, P_corr = ekf_meas_fcn(x_hat_pred, P_pred, y_meas)
x_hat_next, P_next = ekf_time_fcn(x_hat_corr, P_corr, u)

```

It supports exactly one of two starting routes:

- Direct route: provide `x0` and `x0_hat`. If `P0` is omitted, the identity covariance is used.
- Post-initialization route: provide `x_minus_M`, `x0_ref`, `u0_ref`, and a positive `init_steps`. The warm-up phase returns the plant state, observer estimate, and EKF covariance at the start of the training window. If `x_hat_minus_M` is omitted, provide the nominal equilibrium state `x_s`.

For example:

```

dataset = collect_post_initialization_training_data_fast(
    state_fcn,
    output_fcn,
    mpc_fcn,
    ekf_meas_fcn,
    ekf_time_fcn,
    x_ref=x_ref,
    u_ref=u_ref,
    sim_steps=num_sim_steps,
    x_minus_M=x_minus_M,
    x_s=x_s,
    x0_ref=x0_ref,
    u0_ref=u0_ref,
    init_steps=num_init_steps,
    x_min=x_min,
    x_max=x_max,
    u_min=u_min,
    u_max=u_max,
    target_num_trajs=num_training_trajs,
)

```

The collector uses `jax.lax.scan`, `jax.vmap`, and `jax.jit`. Plant, MPC, EKF, covariance, and controller-state values must therefore be compatible with fixed-shape JAX transforms.

References accept one shared constant, one shared trajectory, one constant per trajectory, or one trajectory per rollout. The supported shapes are

(dim,)	one shared constant,
(steps, dim)	one shared trajectory,
(num_trajs, dim)	one constant per trajectory,
(num_trajs, steps, dim)	one trajectory per rollout.

Use a three-dimensional value when a two-dimensional shape would be ambiguous.

The returned dictionary contains accepted list-of-trajectories signals `U`, `Y`, `X_hat`, `X_true`, and `P`; terminal values; initial training-window values; optional initialization histories; and `post_init_v3` metadata. Optional bounds reject invalid trajectories after simulation. By default, the collector also applies a terminal tracking check whose tolerance scales with the reference output and the available measurement-noise standard deviation. Pass `trajectory_filter=None` to disable that check, or pass a callable to replace it with a domain-specific rule. `target_num_trajs` requests a fixed number of accepted trajectories.

10.1 Low-Level Rollout Helpers

The wrapper is built from:

```
simulate_initialization_trajectories_fast(...)
simulate_training_trajectories_fast(...)
```

The first helper returns the warmed-up `x0`, `x0_hat`, and `P0`. The second returns batched training histories and terminal values.

11 Outputs

The fitting routine returns

```
model, training_info
```

where the `training_info` dictionary contains:

- `model`: the fitted model;
- `scaling_info`: `use_scaling`, `u_mean`, `u_gain`, `yyref_mean`, and `yyref_gain`;
- `nz_hidden`: recurrent hidden-state dimension;
- `nu_surrogate`: surrogate control-output dimension;
- `training_num_trajs`: number of training trajectories;
- `validation_num_trajs`: number of validation trajectories, when supplied;
- `selection_mode`: selected parallel-fit scoring path;
- `best_score`: selected candidate score when parallel fitting is enabled;
- `n_jobs`: resolved parallel worker-job limit, or `None` for `jax-sysid`'s default `cpu_count()` behavior;

- `options`: resolved training options;
- `training_timing`: fit timing and solver metadata;
- Lyapunov weight and tube metadata when the default regularizer is used.

12 Save, Load, and Run the Surrogate

The package exposes `save_training_info(...)` and `load_training_info(...)` to reuse a fitted model without retraining. Because the recurrent architecture belongs to user code, recreate the model before loading saved parameters:

```
from ctrlfit import load_training_info, save_training_info

save_training_info(training_info, "results/ctrlfit_model.pkl")
model, training_info = load_training_info(
    "results/ctrlfit_model.pkl",
    model=model,
)
```

Set `auto_save_training_info=True` in `ctrlfit(...)` to save the returned metadata automatically to `ctrlfit_training_info.pkl`.

The following helpers run the fitted surrogate:

```
surrogate_model_step(...)
surrogate_control(...)
simulate_surrogate_closed_loop(...)
plot_comparison_results(...)
```

`surrogate_model_step(...)` evaluates one recurrent-model step, `surrogate_control(...)` applies the stored scaling convention, `simulate_surrogate_closed_loop(...)` runs a plant rollout, and `plot_comparison_results(...)` saves a compact comparison plot.

13 CSTR Examples

The examples use a continuous stirred-tank reactor (CSTR) benchmark; see [3] for background on CSTR process modeling.

The complete workflow

`examples/example_cstr.py`

generates NMPC-EKF expert controller-observer rollouts, filters training, validation, and held-out test splits, trains or loads a Lyapunov-regularized recurrent surrogate, and saves a comparison figure at `examples/results/cstr_ctrlfit.png`.

The complete workflow uses a configurable post-initialization window and a preset CSTR initialization region. To reuse an existing generated dataset, run

`examples/example_cstr_dataset.py`.

Both CSTR workflows write generated artifacts under `examples/results/`:

`cstr_dataset.pkl`

Cached NMPC-EKF trajectory data for the CSTR example. It contains the training, validation, and held-out test splits used by the example scripts. This file lets `example_cstr_dataset.py` skip the data-generation stage.

`cstr_ctrlfit.pkl`

Saved CtrlFit surrogate training result. It contains the fitted surrogate parameters and training metadata. Recreate the recurrent surrogate architecture in the example code before loading this file.

`cstr_ctrlfit.png`

Comparison plot for the original NMPC-EKF controller and the fitted surrogate on a held-out CSTR trajectory.

The `.pkl` files are Python pickle files and should only be loaded from trusted sources.

References

- [1] K. Xie, P. Krupa, and A. Bemporad, “Learning practically stabilizing surrogate controllers approximating nonlinear controller-observer pairs,” manuscript, 2026.
- [2] A. Bemporad, “An L-BFGS-B approach for linear and nonlinear system identification under ℓ_1 and group-Lasso regularization,” *IEEE Transactions on Automatic Control*, vol. 70, no. 7, pp. 4857–4864, 2025. <https://doi.org/10.1109/TAC.2024.3406595>
- [3] B. W. Bequette, *Process Dynamics: Modeling, Analysis and Simulation*, Prentice-Hall, 1998.

14 License

Apache License 2.0

© 2026 Kui Xie